C++ Server Architecture Portfolio Project

서버 구조 흐름 비교 및 시각화 기획 문서

개발 환경: Linux (C++, epoll) / MFC (Windows Client) / TCP + JSON

2026.03

목차 (Table of Contents)

1. 프로젝트 개요

1.1 프로젝트 목적
1.2 여러 서버 구조를 비교하는 이유
1.3 기존 서버 구조의 한계

2. 전체 시스템 구조 설계

2.1 전체 처리 흐름
2.2 Client Protocol / Internal Protocol 개념
2.3 Router 의 역할 (중앙 컨트롤러)
2.4 Session 관리 구조

3. 서버 구조 설계

3.1 서버 1: 요청-응답 구조 서버 (동기 처리)
3.1.1 구조 설명 / 3.1.2 처리 흐름 / 3.1.3 특징 / 3.1.4 장단점 / 3.1.5 사용 목적
3.2 서버 2: epoll 기반 이벤트 루프 서버
3.2.1 구조 설명 / 3.2.2 처리 흐름 / 3.2.3 특징 / 3.2.4 장단점 / 3.2.5 사용 목적
3.3 서버 3: EventBus 기반 서버 (이벤트 흐름 구조)
3.3.1 구조 설명 / 3.3.2 처리 흐름 / 3.3.3 EventBus 설계 / 3.3.4 장단점 / 3.3.5 사용 목적
3.4 서버 4: 혼합 구조 (epoll + EventBus + JobQueue + Worker Thread Pool)
3.4.1 구조 설명 / 3.4.2 처리 흐름 / 3.4.3 JobQueue / 3.4.4 Worker Pool / 3.4.5 장단점 / 3.4.6 사용 목적
3.5 서버 5: AI 처리 포함 서버 (비동기 처리 구조)
3.5.1 구조 설명 / 3.5.2 처리 흐름 / 3.5.3 AI 처리 상세 / 3.5.4 장단점 / 3.5.5 사용 목적

4. 처리 흐름 설계 (Flow)

4.1 전체 흐름 비교
4.2 구조별 상세 흐름 다이어그램

5. 구조 비교 분석

5.1 종합 비교표
5.2 구조 발전 단계 분석

6. 설계 핵심 포인트

6.1 왜 Router 를 공통으로 두었는가
6.2 왜 Service 이후에서 구조를 나눠는가
6.3 왜 AI 는 비동기 처리인가
6.4 핵심 설계 원칙 요약

1. 프로젝트 개요

1.1 프로젝트 목적

본 프로젝트는 단순한 서버 기능 구현이 아니라, 동일한 비즈니스 로직을 서로 다른 서버 아키텍처로 구현하여 각 구조의 처리 흐름을 비교하고 시각화하는 포트폴리오 프로젝트이다.

핵심 목표: 서버 아키텍처 설계 능력을 실제 코드로 증명하는 것이며, 각 구조의 특성과 한계를 실무적 관점에서 분석하고 비교하는 능력을 보여준다.

1.2 여러 서버 구조를 비교하는 이유

현업에서 서버 개발은 단순히 '동작하는 코드'를 작성하는 것이 아니다. 요구사항에 맞는 적절한 아키텍처를 선택하고, 그 선택의 근거를 설명할 수 있어야 한다. 본 프로젝트는 다음을 증명한다:

- 동일 기능을 서로 다른 구조로 구현할 때 발생하는 처리 흐름의 차이를 실제 코드로 보여준다
- 각 구조의 장단점과 적합한 사용 시나리오를 분석한다
- 아키텍처 설계 능력과 구조적 사고력을 포트폴리오로 증명한다
- 단계별 복잡도 증가를 통해 각 기술 요소의 필요성을 설명한다

1.3 기존 서버 구조의 한계

전통적인 서버 구조들은 각각 명확한 한계를 가지고 있으며, 이를 해결하기 위해 점진적으로 복잡한 구조가 발전해 왔다:

구조	한계	발전 방향
Thread-per-Connection	스레드 폭발, 컨텍스트 스위칭 비용	epoll 기반 이벤트 루프
동기 처리	I/O 블로킹으로 인한 처리량 제한	비동기 I/O + 이벤트 구동
모놀리식 처리	기능 확장 시 코드 결합도 증가	EventBus 기반 분리
단순 이벤트 구조	무거운 작업 시 이벤트 루프 블로킹	JobQueue + Worker Thread Pool

본 프로젝트는 이러한 한계들을 단계별로 해결해 나가는 과정을 5 개 서버 구조로 보여준다.

2. 전체 시스템 구조 설계

2.1 전체 처리 흐름

모든 서버 구조는 다음의 공통 처리 흐름을 공유한다. 클라이언트 요청은 반드시 Router 를 거쳐 Service 로 전달되며, Service 이후의 처리 방식이 서버 구조별로 달라진다:

공통 처리 흐름 (Common Flow)

[MFC Client]
TCP 연결 → JSON 패킷 전송

↓

[TCP Socket (epoll)]
수신된 바이트 스트림 → JSON 파싱

↓

[Router]
Client Protocol → Internal Protocol 변환 및 라우팅

↓

[Service]
비즈니스 로직 처리

↓

[← 구조별 분기점 →]
이 지점부터 각 서버 구조의 처리 방식이 달라진다

2.2 Client Protocol / Internal Protocol 개념

클라이언트와 서버 간의 통신에서 프로토콜 변환은 핵심 설계 요소이다:

구분	Client Protocol	Internal Protocol
형식	JSON 문자열	C++ enum class
예시	{"type":"LOGIN","id":"user01","pw":"1234"}	InternalType::LOGIN
사용 위치	네트워크 구간 (Client ↔ Server)	서버 내부 처리 구간
변환 시점	Router 에서 변환	Service/EventBus 에서 사용
목적	클라이언트 호환성 보장	타입 안전성 + 성능 최적화

이 변환 구조를 통해 클라이언트 통신 포맷(JSON)이 변경되더라도 서버 내부 로직은 영향을 받지 않는다. Router 가 변환 책임을 전담하므로, Service 계층은 프로토콜 변경에 대해 완전히 분리된다.

2.3 Router 의 역할 (중앙 컨트롤러)

Router 는 모든 서버 구조에서 공통으로 사용되는 중앙 컨트롤러이다. 다음의 책임을 수행한다:

1. JSON 패킷 파싱 및 유효성 검증
2. Client Protocol("type" 문자열) → Internal Protocol(enum) 변환
3. Internal Protocol 에 따른 대상 Service 함수 호출 (dispatch)
4. Session 관리 (클라이언트 fd ↔ 사용자 정보 매핑)
5. 응답 JSON 생성 및 클라이언트 전송

설계 의도: Router 를 공통 계층으로 두면, 서버 구조가 변경되어도 입구(Router)와 출구(응답)는 동일하다. 이를 통해 5 개 서버 구조의 '처리 경로'만 순수하게 비교할 수 있다.

2.4 Session 관리 구조

Session 은 클라이언트 연결 상태를 관리하는 구조체이다. TCP 소켓의 file descriptor(fd)를 키로 사용하여 클라이언트별 상태를 추적한다:

필드	타입	설명
fd	int	TCP 소켓 파일 디스크립터
user_id	std::string	로그인된 사용자 ID
state	enum	CONNECTED / AUTHENTICATED / DISCONNECTED
recv_buffer	std::string	수신 버퍼 (불완전 패킷 처리용)
last_active	time_point	마지막 활동 시간 (timeout 검사용)

3. 서버 구조 설계

3.1 서버 1: 요청-응답 구조 서버 (동기 처리)

3.1.1 구조 설명

가장 기본적인 서버 구조이다. 클라이언트의 요청을 받으면, 해당 요청을 즉시 처리하고 응답을 반환한다. 모든 처리가 단일 스레드에서 순차적으로 이루어지며, 요청 A 의 처리가 완료되어야 요청 B 의 처리가 시작된다.

단, epoll 을 사용하여 I/O 멀티플렉싱은 수행하지만, 비즈니스 로직 자체는 동기적으로 처리한다. 즉, epoll 이 이벤트를 감지하면 해당 요청을 받아 Router 가 즉시 Service 를 호출하고, Service 가 처리를 완료한 후 응답을 반환한다.

3.1.2 처리 흐름

서버 1: 요청-응답 구조

[Client]
JSON 요청 전송

↓

[epoll_wait()]
이벤트 감지 (어떤 fd 에 데이터가 왔는지)

↓

[Router]
JSON 파싱 → Internal Protocol 변환 → Service 직접 호출

↓

[Service]
비즈니스 로직 처리 (동기)

↓

[Response]
JSON 응답 생성 → Client 전송

3.1.3 특징

항목	내용
처리 방식	동기 (Synchronous)

스레드 모델	단일 스레드 (epoll 루프 내 직접 처리)
흐름 제어	요청 → 처리 → 응답 (순차적)
I/O 모델	epoll (non-blocking accept/recv) + 동기 비즈니스 로직

3.1.4 장단점

장점	단점
구조가 단순하여 이해하기 쉽다	Service 가 블로킹되면 전체 서버 멈춤
디버깅과 테스트가 용이하다	동시 처리 불가 (복수 요청 순차 대기)
레이스 컨디션 발생 불가	무거운 작업 시 epoll 루프 블로킹

3.1.5 사용 목적

서버 구조의 기본 형태를 이해하기 위한 기준점(baseline)으로 사용한다. 이후 구조들이 '왜 필요한지'를 설명하는 출발점이 된다.

3.2 서버 2: epoll 기반 이벤트 루프 서버

3.2.1 구조 설명

서버 1 의 한계인 '단일 클라이언트 처리 중 블로킹' 문제를 해결한다. epoll 을 이벤트 루프의 핵심으로 사용하여 복수의 클라이언트를 단일 스레드에서 효율적으로 처리한다. epoll 이 이벤트를 감지하면, 해당 이벤트에 대한 처리를 즉시 수행하고 다음 이벤트로 넘어간다.

thread-per-connection 모델을 사용하지 않으며, 단일 스레드가 epoll_wait()를 통해 모든 클라이언트의 I/O 이벤트를 멀티플렉싱한다. 이는 수천 개의 동시 연결을 효율적으로 처리할 수 있는 구조이다.

3.2.2 처리 흐름

서버 2: epoll 기반 이벤트 루프

[epoll_wait()]
복수 fd 이벤트 동시 감지

↓

[Event Dispatch]
이벤트 타입별 분기 (accept / recv / error)

↓

[Router]
recv 이벤트 → JSON 파싱 → 라우팅

↓

[Service]
비즈니스 로직 처리 (non-blocking 작업만)

↓

[Response + Loop]
응답 전송 → epoll_wait()로 복귀

3.2.3 특징

항목	내용
처리 방식	이벤트 구동 (Event-Driven)
스레드 모델	단일 스레드 + epoll 멀티플렉싱
흐름 제어	이벤트 루프 (epoll_wait → dispatch → 처리 → 반복)

I/O 모델	완전한 non-blocking I/O

3.2.4 장단점

장점	단점
단일 스레드로 수천 개 연결 처리 가능	무거운 작업이 이벤트 루프를 블로킹할 수 있음
컨텍스트 스위칭 비용 없음	CPU 집약적 작업 시 다른 클라이언트 응답 지연
메모리 사용량 최소화	모든 처리가 non-blocking 이어야 하는 제약

3.2.5 사용 목적

I/O 멀티플렉싱의 효율성을 보여주며, 이벤트 루프 기반 서버의 기본 패턴을 이해하는 데 사용한다. '무거운 작업을 어떻게 처리할 것인가'라는 문제를 제기하는 단계이다.

3.3 서버 3: EventBus 기반 서버 (이벤트 흐름 구조)

3.3.1 구조 설명

서버 2 의 구조에서 Service 가 Router 에 직접 결합되는 문제를 해결한다. EventBus 를 도입하여 Service 와 처리 계층을 완전히 분리한다.

EventBus 는 publish/subscribe 패턴을 사용한다. Service 는 이벤트를 EventBus 에 publish 하고, 해당 이벤트에 subscribe 된 핸들러가 처리를 수행한다. Service 는 누가 처리하는지 모른다. 이를 통해 Service 간 결합도가 극적으로 낮아진다.

3.3.2 처리 흐름

서버 3: EventBus 기반

[Client 요청]
JSON 요청 수신

↓

[Router]
JSON 파싱 → Internal Protocol 변환

↓

[Service]
비즈니스 로직 판단 → EventBus 에 이벤트 publish

↓

[EventBus]
이벤트 타입별 등록된 handler 호출 (subscribe)

↓

[Handler]
실제 처리 수행 → 결과 반환

↓

[Response]
JSON 응답 → Client 전송

3.3.3 EventBus 설계 상세

구성 요소	설명
Event	이벤트 타입(enum) + 데이터(페이로드) 구조체

subscribe(type, handler)	특정 이벤트 타입에 핸들러 등록
publish(event)	이벤트 발행 → 등록된 모든 핸들러 호출
Handler	std::function<void(const Event&)> 형태

3.3.4 장단점

장점	단점
구성 요소 간 결합도 최소화	이벤트 흐름 추적이 복잡해질 수 있음
새로운 기능 추가 시 handler 만 등록하면 됨	모든 처리가 여전히 단일 스레드에서 실행됨
테스트가 용이함 (handler 단위 테스트 가능)	무거운 handler 가 전체 서버를 블로킹할 수 있음

3.3.5 사용 목적

이벤트 기반 아키텍처의 핵심 개념인 publish/subscribe 패턴을 시연한다. 구성 요소 간 결합도를 낮춰 확장성을 확보하는 구조이며, '무거운 처리를 비동기로 분리해야 한다'는 문제를 제기하는 단계이다.

3.4 서버 4: 혼합 구조 (epoll + EventBus + JobQueue + Worker Thread Pool)

3.4.1 구조 설명

서버 3 의 '단일 스레드에서 모든 처리' 한계를 해결한다. EventBus 를 통해 이벤트를 발행하되, 무거운 작업은 JobQueue 를 통해 Worker Thread Pool 로 위임한다. 이를 통해 epoll 스레드는 절대 블로킹되지 않으며, 이벤트 처리와 비즈니스 로직이 완전히 분리된다.

핵심 원칙: Service 는 절대 Worker 를 직접 호출하지 않는다. 반드시 EventBus 를 통해 이벤트를 발행하고, EventBus 의 handler 가 JobQueue 에 작업을 등록하며, Worker Thread 가 이를 가져가 처리한다.

3.4.2 처리 흐름

서버 4: 혼합 구조

[Client 요청]
JSON 요청 수신

↓

[epoll_wait()]
복수 fd 이벤트 감지 (non-blocking)

↓

[Router]
JSON 파싱 → Internal Protocol 변환 → Service 호출

↓

[Service]
비즈니스 판단 → EventBus 에 이벤트 publish

↓

[EventBus]
handler 호출 → JobQueue 에 작업 등록 (enqueue)

↓

[JobQueue]
thread-safe 대기열 (mutex + cond_var, 크기 제한)

↓

[Worker Thread Pool]
고정 개수 스레드가 작업 가져가 처리 (dequeue)

↓

[Response]
Worker → 응답 JSON 생성 → Client 전송

3.4.3 JobQueue 설계 상세

구성 요소	설명
동기화	std::mutex + std::condition_variable
대기열	std::queue<Job> (선입선출)
크기 제한	최대 큐 크기 설정 (backpressure)
overflow 처리	큐 가득 찼 때 reject 또는 drop 처리
enqueue()	epoll 스레드 / EventBus handler 가 호출
dequeue()	Worker Thread 가 호출 (blocking wait)

3.4.4 Worker Thread Pool 설계

구성 요소	설명
스레드 수	고정 개수 (e.g., 4 개)
동작 방식	JobQueue 에서 작업을 가져가 처리 (dequeue + execute)
스레드 생명주기	서버 시작 시 생성 → 서버 종료 시 join
종료 처리	stop flag + condition_variable notify_all

3.4.5 장단점

장점	단점
epoll 스레드가 절대 블로킹되지 않음	구조가 복잡하여 디버깅 난이도 증가
무거운 작업과 경량 작업 분리	동기화 처리(mutex) 필요
Worker 수 조절로 처리량 제어 가능	JobQueue overflow 시 요청 손실 가능성
확장성 우수	thread-safe 처리 비용 (lock contention)

3.4.6 사용 목적

실무 환경에서 사용 가능한 서버 구조의 완성형을 보여준다. I/O 처리와 비즈니스 로직이 완전히 분리되어 있으며, 각 구성 요소가 독립적으로 확장 가능한 구조이다.

3.5 서버 5: AI 처리 포함 서버 (비동기 처리 구조)

3.5.1 구조 설명

서버 4 의 혼합 구조를 기반으로, AI/ML 추론과 같은 외부 비동기 처리를 추가한 구조이다. AI 처리는 수백 ms 에서 수 초가 걸릴 수 있으므로, 반드시 비동기로 처리해야 한다.

핵심은 AI 요청이 발생하면 즉시 응답하지 않고, '처리 중' 상태를 먼저 반환한 후, AI 처리가 완료되면 결과를 별도로 전송하는 2 단계 응답 구조이다.

3.5.2 처리 흐름

서버 5: AI 처리 포함 비동기 구조

[Client 요청]
AI 처리 요청 (e.g., {"type":"AI_ANALYZE","data":"..."})

↓

[Router → Service]
JSON 파싱 → Internal Protocol → AI Service 호출

↓

[Service → EventBus]
AI_REQUEST 이벤트 publish + 클라이언트에 '처리 중' 응답

↓

[EventBus → JobQueue]
AI 처리 작업을 JobQueue 에 enqueue

↓

[Worker Thread]
AI 추론 처리 수행 (비동기, 장시간 소요)

↓

[AI 처리 완료]
AI_COMPLETE 이벤트 publish

↓

[Response Handler]
결과 JSON 생성 → Client 에 비동기 응답 전송

3.5.3 AI 처리 흐름 상세

단계	처리 내용	스레드
1 차 응답	클라이언트에 '처리 중' 상태 즉시 반환	epoll 스레드
AI 대기열	JobQueue 에 AI 작업 등록	epoll 스레드
AI 처리	실제 AI 추론 수행	Worker Thread
2 차 응답	AI 결과를 클라이언트에 비동기 전송	Worker Thread → epoll

3.5.4 장단점

장점	단점
AI 처리 중에도 서버 정상 운영	가장 복잡한 구조
2 단계 응답으로 클라이언트 UX 개선	클라이언트 측에서 비동기 응답 처리 필요
비동기 처리의 본질적 패턴 시연	timeout 및 에러 처리 복잡도 증가
외부 API 연동 구조 확장 가능	동시 처리 제한 (Worker 수 = 최대 병렬 AI 요청)

3.5.5 사용 목적

실무에서 점점 더 중요해지는 AI/ML 연동 시나리오를 보여준다. 비동기 처리, 2 단계 응답, timeout 처리 등 실제 프로덕션 환경에서 필요한 패턴을 모두 포함한다.

4. 처리 흐름 설계 (Flow)

4.1 전체 흐름 비교

다음은 5 개 서버 구조의 처리 흐름을 한 눈에 비교한 것이다. 공통 구간(Client → Router → Service)과 구조별 분기 구간을 명확히 구분한다.

단계	서버 1	서버 2	서버 3	서버 4	서버 5
1. 수신	epoll	epoll	epoll	epoll	epoll
2. 파싱	Router	Router	Router	Router	Router
3. 변환	Protocol	Protocol	Protocol	Protocol	Protocol
4. 처리	Service 직접	Service 직접	EventBus	EventBus	EventBus
5. 실행	-	-	Handler	JobQueue	JobQueue
6. 작업자	동일 스레드	동일 스레드	동일 스레드	Worker	Worker
7. 응답	즉시	즉시	즉시	즉시	2 단계

4.2 구조별 상세 흐름 다이어그램

서버 1 (요청-응답):

Client → [epoll] → [Router: JSON→enum] → [Service: 동기 처리] → [Response] → Client

서버 2 (epoll 이벤트 루프):

Client → [epoll_wait 루프] → [Event Dispatch] → [Router] → [Service: non-blocking 처리] → [Response] → [epoll_wait 복귀]

서버 3 (EventBus):

Client → [epoll] → [Router] → [Service: EventBus.publish()] → [EventBus: handler 호출] → [Handler 처리] → [Response]

서버 4 (혼합):

Client → [epoll] → [Router] → [Service: EventBus.publish()] → [EventBus → JobQueue.enqueue()] → [Worker Thread: dequeue + 처리] → [Response]

서버 5 (AI 비동기):

Client → [epoll] → [Router] → [Service: EventBus.publish() + 1 차 응답('처리중')] → [EventBus → JobQueue] → [Worker: AI 처리] → [2 차 응답(AI 결과)]

5. 구조 비교 분석

5.1 종합 비교표

기준	서버 1 요청-응답	서버 2 epoll 루프	서버 3 EventBus	서버 4 혼합 구조	서버 5 AI 비동기
동기/비동기	동기	동기	동기	비동기	비동기
처리 방식	순차적	이벤트 루프	이벤트+pub/sub	이벤트+Worker	이벤트+Worker+2단계
확장성	★☆☆☆☆	★★☆☆☆	★★★☆☆	★★★★☆	★★★★★
성능 (동시처리)	★☆☆☆☆	★★★☆☆	★★★☆☆	★★★★☆	★★★★★
구조 복잡도	★☆☆☆☆	★★☆☆☆	★★★☆☆	★★★★☆	★★★★★
결합도	높음	높음	낮음	낮음	낮음
I/O 모델	epoll+동기	epoll+NB	epoll+NB	epoll+NB	epoll+NB
스레드	단일	단일	단일	멀티	멀티
블로킹 위험	높음	중간	중간	낮음	낮음

5.2 구조 발전 단계 분석

각 서버 구조는 이전 구조의 한계를 해결하며 점진적으로 발전한다:

전환	해결한 문제	도입한 기술
서버 1 → 2	단일 클라이언트 처리 중 블로킹	epoll 이벤트 루프
서버 2 → 3	Service ↔ Router 간 강한 결합	EventBus (pub/sub 패턴)
서버 3 → 4	단일 스레드에서 무거운 처리 블로킹	JobQueue + Worker Thread Pool
서버 4 → 5	장시간 소요 작업 시 클라이언트 대기	비동기 2 단계 응답

6. 설계 핵심 포인트

6.1 왜 Router 를 공통으로 두었는가

설계 의도: 서버 구조가 변경되어도 클라이언트와의 인터페이스는 동일하게 유지한다.

Router 는 모든 서버 구조에서 동일한 역할을 수행한다. 이를 통해:

6. 클라이언트는 서버 내부 구조 변경을 인지할 필요가 없다 (동일한 MFC 클라이언트로 5 개 서버 모두 테스트 가능)
7. 5 개 서버의 차이점이 "Service 이후의 처리 경로"에만 집중되어 비교가 명확해진다
8. Protocol 변환 로직이 한 곳에서 관리되어 유지보수성이 높다

6.2 왜 Service 이후에서 구조를 나눠는가

설계 의도: 비즈니스 로직의 '실행 방식'을 구조적으로 분리하여 비교하기 위함이다.

Service 는 '무엇을 할 것인가'를 결정하는 계층이다. Service 이후의 구조는 '어떻게 실행할 것인가'를 결정한다. 이 분리를 통해:

- 동일한 비즈니스 로직(Service)이 서로 다른 실행 방식(동기/비동기/이벤트/Worker)으로 처리되는 것을 보여줄 수 있다
- 구조 변경이 Service 코드에 영향을 주지 않아 테스트가 용이하다
- '실행 방식'만 교체하면 새로운 구조를 실험할 수 있다 (플러그인 방식)

6.3 왜 AI 는 비동기 처리인가

설계 의도: AI 처리는 본질적으로 비동기이어야 하며, 이를 통해 서버의 응답성을 유지한다.

AI/ML 추론은 다음과 같은 특성을 가진다:

9. 처리 시간 예측 불가: 수백 ms 에서 수 초까지 다양한 처리 시간
10. 외부 API 의존: 네트워크 지연, 서버 장애 등 외부 요인에 영향을 받음
11. CPU/GPU 집약적: 처리 중 다른 클라이언트 응답을 블로킹할 수 있음
12. 결과 즉시 필요 없음: 사용자가 대기할 수 있는 유형의 작업

따라서 AI 처리를 동기적으로 처리하면 해당 요청이 완료될 때까지 다른 모든 클라이언트의 요청이 블로킹된다. 비동기 2 단계 응답 구조를 통해 이 문제를 해결한다:

단계	처리	목적
1 차 응답	즉시 "processing" 상태 반환	클라이언트 UX 유지 (응답 없음 방지)
AI 처리	Worker Thread 에서 비동기 수행	epoll 스레드 블로킹 방지

2 차 응답	AI 결과 전송	실제 결과 전달

6.4 핵심 설계 원칙 요약

원칙	설명	적용 위치
epoll 스레드 보호	epoll 스레드는 절대 blocking 작업을 하지 않는다	서버 2, 3, 4, 5
Service → EventBus 간접호출	Service 는 Worker 를 직접 호출하지 않는다	서버 3, 4, 5
JobQueue thread-safe	mutex + condition_variable 로 동기화	서버 4, 5
JobQueue backpressure	큐 크기 제한 + overflow 처리	서버 4, 5
Worker 고정 수	Worker Thread 는 고정 개수만 사용	서버 4, 5
Protocol 변환 분리	JSON ↔ enum 변환은 Router 에서만 수행	모든 서버

End of Document

본 문서는 C++ 서버 아키텍처 포트폴리오 프로젝트의 기획 문서입니다.